

## Introduction

---

Every backend engineer eventually writes a bug that looks harmless until money — or data — disappears. It usually looks something like this: you update one row, then update a second row, and somewhere between those two statements, something goes wrong. A network blip. A constraint violation. A server crash. The first update already happened. The second one didn't. Now your database is lying to you.

This is one of those bugs that doesn't show up in code review, because the code reads perfectly fine line by line. It only shows up in production, usually during a demo, usually involving someone's money. That's exactly why SQL transactions and the ACID properties are a recurring topic in backend interviews — not because interviewers care if you memorized four letters, but because they want to know if you understand what "correct" means when multiple writes have to succeed or fail together.

By the end of this article, you'll understand why transactions exist, what each ACID property actually guarantees, and — just as importantly — when you don't need to reach for one.

## Problem Overview

---

Imagine a simple bank transfer: move \$100 from Account A to Account B. Naively, this is two separate SQL statements:

1. Subtract \$100 from Account A.
2. Add \$100 to Account B.

Run individually, each statement is valid SQL and each one succeeds or fails on its own. The problem is that *together* they represent a single logical operation. If the first statement runs and the second one fails — because the server crashes, the connection drops, or Account B doesn't exist — you're left with a database that already took the money out of Account A but never put it into Account B. The money isn't lost in some abstract sense; it's just gone, and no query can find it, because it was never written anywhere.

This is the core problem a transaction solves: some sequences of writes only make sense as a single unit. Either all of them happen, or none of them do. There is no acceptable middle state.

## Example

---

Let's say Account A has a balance of \$500 and Account B has a balance of \$200.

### Without a transaction:

```
UPDATE accounts SET balance = balance - 100 WHERE id = 'A';  
-- server crashes here  
UPDATE accounts SET balance = balance + 100 WHERE id = 'B';
```

Result: Account A now has \$400. Account B still has \$200. Total money in the system: \$600, down from \$700. \$100 evaporated.

### With a transaction:

```
BEGIN;  
UPDATE accounts SET balance = balance - 100 WHERE id = 'A';  
UPDATE accounts SET balance = balance + 100 WHERE id = 'B';  
COMMIT;
```

If the crash happens before `COMMIT`, the database discards the first update automatically when it restarts. Account A is back to \$500, exactly as if the transfer never started. Total money in the system: still \$700.

That's the entire value proposition in one comparison. Nothing clever, just a guarantee that half-finished work doesn't get to count.

## Intuition

---

Here's how an engineer arrives at this solution without ever having heard the word "ACID." You start by asking: what does it *mean* for this operation to be correct? Not "did each SQL statement execute without an error," but "does the world make sense after this operation, no matter what interrupts it?"

Once you frame it that way, you realize individual statements are the wrong unit of correctness. The unit of correctness is the whole transfer. Debiting one account without crediting the other isn't a partial success — it's a failure that happens to look like a success, because the database didn't complain.

The next question is: how would you *implement* "all or nothing" yourself, if the database didn't offer it? You'd probably reach for some kind of log — write down what you're about to do before you do it, so that if you get interrupted, you can either finish the job or undo it. That's not a hack; that's essentially what the database's transaction log does under the hood. Recognizing that "all or nothing" requires a durable record of intent is the entire insight. Once you see that, `BEGIN` / `COMMIT` / `ROLLBACK` stop looking like syntax to memorize and start looking like the obvious tool for the job.

From there, ACID isn't four unrelated rules — it's four different ways the "half-finished state" problem can sneak back in, even after you've wrapped things in a transaction:

- What if the transaction itself is interrupted mid-way? → **Atomicity**
- What if a completed transaction leaves the data in a state that violates your business rules? → **Consistency**
- What if someone else reads the data while your transaction is still running? → **Isolation**
- What if the transaction completes, gets confirmed, and then the server loses power? → **Durability**

Each property is closing off one more way the "all or nothing" promise could quietly break.

## Brute Force Solution

---

The "brute force" approach here is to not use a transaction at all, and instead try to handle failure manually in application code:

```
def transfer_naive(conn, from_id, to_id, amount):
    conn.execute("UPDATE accounts SET balance = balance - %s WHERE id = %s", (amount, from_id))
    try:
        conn.execute("UPDATE accounts SET balance = balance + %s WHERE id = %s", (amount, to_id))
    except Exception:
        # try to "undo" the first update manually
        conn.execute("UPDATE accounts SET balance = balance + %s WHERE id = %s", (amount, from_id))
```

**Idea:** run the first update, and if the second one fails, run a compensating update to reverse the first.

**Advantages:** doesn't require understanding transactions; looks like it works in the happy path and in simple failure tests.

**Disadvantages:** - If the process crashes between the two updates (not just an exception, but a full crash), the compensating code never runs at all. You're back to square one. - Concurrent readers can see the half-updated state — Account A already debited, Account B not yet credited — and make decisions based on money that's temporarily nowhere. - You're re-implementing rollback logic by hand, badly, for one specific case, when the database already does this correctly for the general case.

**Time complexity:** O(1) extra queries in the failure path — cheap to run, expensive to trust. **Space complexity:** O(1) — no additional storage, which is exactly the problem; there's no durable record of intent to recover from a hard crash.

## Optimal Solution

The optimal solution is to let the database do what it's already built to do: wrap the related writes in a transaction and let atomicity handle the failure case for you.

```

BEGIN
  |
  ▼
UPDATE Account A (-100)  ← written to transaction log, not yet final
  |
  ▼
UPDATE Account B (+100)  ← written to transaction log, not yet final
  |
  ▼
COMMIT → both changes become permanent and visible, together
  |
  ▼ (if anything fails before COMMIT)
ROLLBACK → both changes are discarded, as if nothing happened

```

The key shift in thinking: you stop treating `UPDATE` statements as immediately final. Inside a transaction, they're provisional — recorded, but not yet real from the outside world's perspective. `COMMIT` is the single moment where "provisional" becomes "permanent," for every statement in the transaction at once. `ROLLBACK` (explicit, or automatic on crash) throws away everything provisional.

This is also where isolation matters: while the transaction is open, other connections querying Account A or Account B don't see the in-progress debit or credit. They see either the pre-transfer state or the post-transfer state — never the moment in between where \$100 is debited but not yet credited.

## Python Code

```

def transfer(conn, from_id: str, to_id: str, amount: int) -> None:
    """Move `amount` from one account to another as a single atomic unit."""
    with conn: # commits on success, rolls back automatically on exception
        conn.execute(
            "UPDATE accounts SET balance = balance - %s WHERE id = %s",
            (amount, from_id),
        )
        conn.execute(
            "UPDATE accounts SET balance = balance + %s WHERE id = %s",
            (amount, to_id),
        )

```

## Code Walkthrough

- `with conn:` — most Python DB-API drivers (psycopg2, sqlite3) treat entering this context manager as starting a transaction. If the block finishes without an exception, the driver issues `COMMIT`. If an exception is raised anywhere inside, it issues `ROLLBACK` automatically. This mirrors `BEGIN ... COMMIT` at the SQL level without you having to write it explicitly.
- The first `execute` debits `from_id`. Nothing is permanent yet — it's recorded, provisional.
- The second `execute` credits `to_id`. If `to_id` doesn't exist, the database driver raises an exception (a foreign key violation or similar).
- Because the exception happens inside the `with` block, the context manager rolls back everything — including the first update — before the exception propagates further.
- Parameterized queries ( `%s` placeholders) are used instead of string formatting to avoid SQL injection — unrelated to transactions, but never skip it.

## Dry Run

---

Starting balances: A = \$500, B = \$200. Call `transfer(conn, 'A', 'B', 100)`.

1. `with conn:` opens a transaction.
2. `UPDATE accounts SET balance = balance - 100 WHERE id = 'A'` → A's balance is provisionally \$400.
3. `UPDATE accounts SET balance = balance + 100 WHERE id = 'B'` → B's balance is provisionally \$300.
4. No exception was raised, so the `with` block exits normally and issues `COMMIT`.
5. Both changes become permanent and visible at once: A = \$400, B = \$300.

Now suppose `to_id = 'C'` and Account C doesn't exist:

1. `with conn:` opens a transaction.
2. A's balance is provisionally debited to \$400.
3. The insert into C fails with a foreign key or constraint error, raising an exception.
4. The `with` block catches the exception on exit and issues `ROLLBACK`.
5. A's balance reverts to \$500. Nothing was ever visible to other queries in between.

## Complexity Analysis

---

**Time:**  $O(1)$  additional overhead per statement for logging, plus the cost of holding any locks acquired during the transaction — negligible for a two-statement transfer like this. The number of statements executed is the same with or without the transaction; you're not doing more work, just grouping it correctly.

**Space:**  $O(k)$  where  $k$  is the number of statements in the transaction, since the database needs to keep a record (write-ahead log) of each provisional change until commit or rollback. For a transfer between two accounts, that's a constant, small amount of extra bookkeeping — trivial compared to the cost of a silent data corruption bug.

These bounds hold because a transaction doesn't change the algorithm — it changes the *guarantee* around an algorithm you were already going to run.

## Alternative Solutions

---

**Application-level compensation (sagas):** for distributed systems where a single database transaction can't span all the writes (e.g., debiting a bank account in one microservice and crediting one in another), you sometimes can't use `BEGIN / COMMIT` at all. The alternative is a saga — each step has a corresponding compensating action, and if a later step fails, you explicitly run the compensations for everything that already succeeded. This is strictly harder to get right than a database transaction and should only be used when a single transaction genuinely isn't possible.

**Optimistic locking with version columns:** for high-concurrency systems, some engineers avoid holding transactions open for coordination and instead use a version number on rows, checking it hasn't changed before committing. This addresses isolation-related contention rather than atomicity, and is a separate concern from the crash-safety problem this article focuses on.

For a single-database bank transfer like the one here, a plain transaction is simpler and more reliable than either alternative.

## Edge Cases

---

- **Empty transaction (no statements between BEGIN and COMMIT):** legal, but a no-op — nothing to commit or roll back.
- **Transferring to a non-existent account:** should fail the second statement and roll back the first — verify your foreign key constraints actually enforce this, or the "failure" won't happen and you'll silently create an orphaned credit.
- **Transferring the maximum representable amount:** watch for integer overflow if you're storing balances as fixed-width integers; this isn't a transaction concern but will corrupt data just as silently.
- **Debiting more than the account's balance:** a transaction won't catch this on its own — you need a `CHECK (balance >= 0)` constraint or application-level validation, otherwise ACID will happily commit a negative balance because nothing told it that's invalid.

- **Nested transactions:** most databases don't support true nested transactions; calling `BEGIN` inside an open transaction either errors or is a no-op, depending on the driver — don't assume savepoints and nested `BEGIN` behave the same way.

## Common Mistakes

---

1. **Assuming a network-level retry is the same as a rollback.** Retrying a failed request without knowing whether the first statement already committed can double-apply a write.
2. **Wrapping read-only queries in transactions "just in case."** This adds lock overhead for no correctness benefit — transactions are for related *writes*, not defensive habit.
3. **Believing `COMMIT` means "fast" rather than "durable."** A successful commit means the data survived a crash, not that it was written quickly — these are different guarantees.
4. **Holding a transaction open across a slow external call** (an API request, a user prompt) while inside `BEGIN / COMMIT`. This holds locks far longer than necessary and can block every other write to those rows.
5. **Forgetting that a transaction protects against crashes, not against bad business logic.** If your consistency rules (constraints, checks) are missing, a transaction will commit an invalid state just as reliably as a valid one.

## Interview Questions

---

- What's the difference between atomicity and isolation, and can you give an example where one holds but the other doesn't?
- What are SQL isolation levels ( `READ COMMITTED` , `REPEATABLE READ` , `SERIALIZABLE` ), and what specific anomaly does each one prevent?
- How does a database implement rollback internally — what does the write-ahead log actually store?
- When would you *avoid* wrapping something in a transaction?
- How do transactions behave differently across multiple database connections versus multiple microservices?

## Similar Problems

---

- **LeetCode-style database design questions involving multi-table consistency** — any problem requiring updates across related tables (e.g., inventory and orders) touches the same atomicity concerns.

- **Distributed systems: two-phase commit** — the multi-node generalization of the same all-or-nothing problem this article covers for a single database.
- **Idempotency key design in payment APIs** — a related but distinct problem: making sure a retried request doesn't apply twice, which pairs naturally with transactional writes.

## Key Takeaways

---

A transaction groups related writes into a single all-or-nothing unit, because some operations only make sense together — debiting one account and crediting another is either both-or-neither, with no valid state in between. ACID is the four-part promise underneath that grouping: atomicity guarantees the all-or-nothing behavior itself, consistency guarantees the result never violates your data's rules, isolation guarantees concurrent transactions don't see each other's half-finished work, and durability guarantees a committed transaction survives a crash. The engineering judgment call isn't whether to use transactions — it's recognizing which writes are actually coupled and wrapping exactly those, no more and no less.

## Watch the Video

---

For the full walkthrough — including a live demonstration of the failure case and the fix — watch the video here: {LINK}

## About the Series

---

This lesson is part of the **Fun with Learning Technology** series, where real backend bugs are used as the entry point into the concepts behind them, instead of teaching theory in isolation. Each lesson starts with something that actually breaks in production and works backward to the fundamentals that explain why — and how to prevent it next time.

## Call To Action

---

If this saved you from a future 2am debugging session, hit like on the video and subscribe on YouTube so the next lesson finds you. For deeper write-ups like this one delivered straight to your inbox, subscribe to the Substack. And if you've hit this exact bug before, drop a comment — I'd genuinely like to hear how you found it.

---



## Quick Review

---

**What's the pattern trigger for wrapping SQL in a transaction?**

When multiple related updates must all succeed or all fail together — e.g. money transfers, multi-table writes.

**Name the four ACID properties in order.**

Atomicity, Consistency, Isolation, Durability.

**What does Atomicity guarantee if step 2 of 3 fails?**

The whole transaction rolls back — no partial writes persist, as if nothing happened.

**Trickiest bug: what happens if you forget COMMIT/ROLLBACK on failure?**

Connection may hold an open transaction, locking rows and silently blocking other queries indefinitely.

**Isolation vs Durability — what's the difference?**

Isolation hides in-progress transactions from concurrent reads; Durability ensures committed data survives a crash.

**Follow-up: why does a failed transfer update need a transaction, not just a try/catch?**

Try/catch alone doesn't undo the first UPDATE already sent to the DB — only a transaction's ROLLBACK reverts committed-in-progress statements.

---

sql Lesson 28: Transactions and ACID Properties (A transaction is a logical unit of work that ensures a sequence of database operations is treated as a single, indivisible action. These properties are critical because they prevent data corruption during system failures or concurrent access scenarios by guaranteeing state consistency. You should utilize transactions whenever multiple related updates must either succeed together or fail entirely to maintain the integrity of your business data.) — free Python cheat sheet